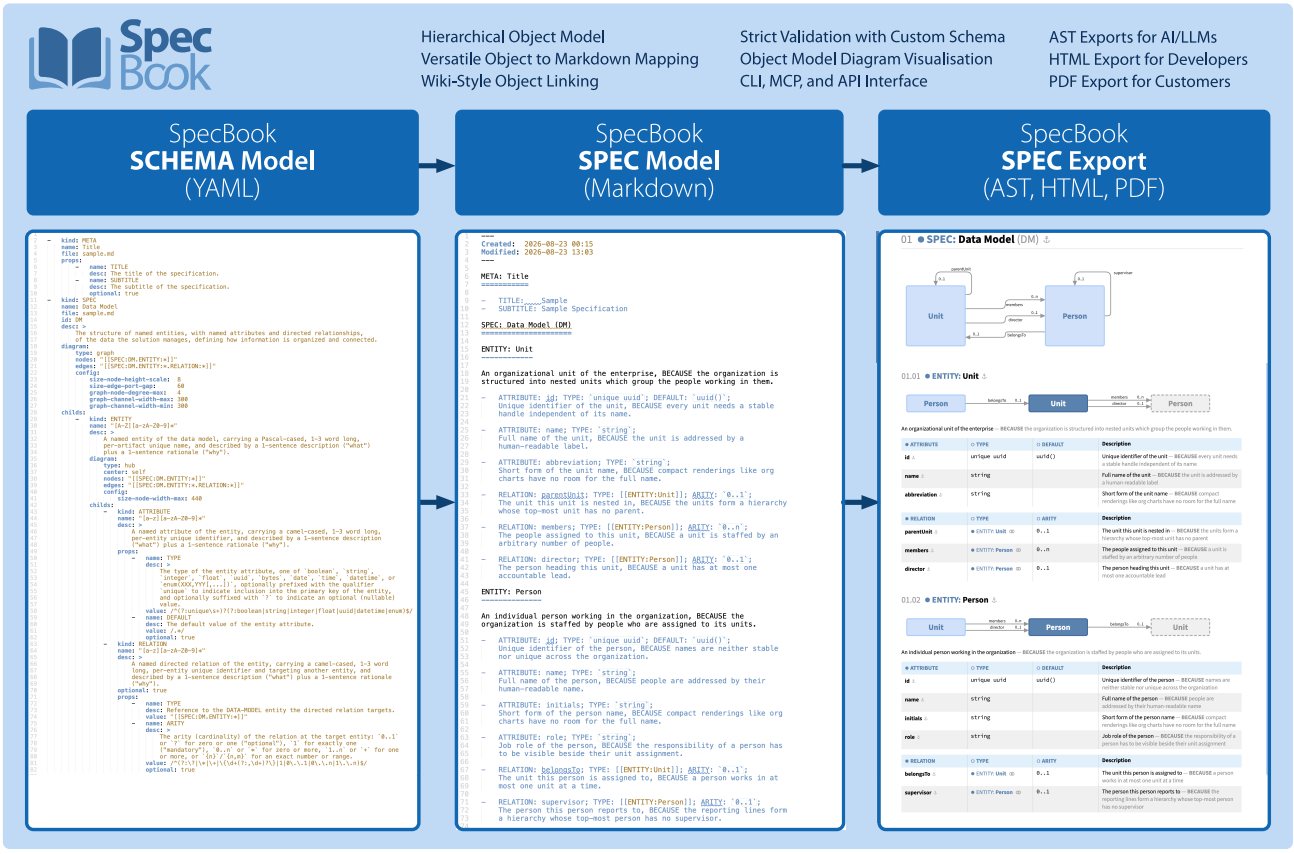


Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

Modern Software Specification

Intellectual Content: Version 1.0.0 (2026-04-28) Authored 2026 by Dr. Ralf D. Engelken
Graphical Illustration: Version 1.0.0 (2026-04-28) Copyright © 2026 Dr. Ralf D. Engelken (https://engelken.ai/cv/). All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.



Im modernen Software Engineering wird eine Software-Spezifikation zunehmend als strukturiertes Modell statt als frei formuliertes Dokument behandelt. Eine Spezifikation wird zu einem hierarchischen Graphen typisierter Objekte wie Entitäten, Attribute und Relationen. Sie wird in einem Klartextformat wie **Markdown** verfasst, und ihre Objekte sind über explizite Querverweise miteinander verknüpft. Dieser **Specification-as-Code-Ansatz** stellt Spezifikationen unter Versionskontrolle, macht Änderungen als “Diffs” nachvollziehbar und hält jede Information an genau einer Stelle.

Eine zentrale Idee ist die Trennung von Regeln und Inhalt. Ein Schema, das Metamodell, legt fest, welche Arten von Objekten eine Spezifikation enthalten darf, wie diese verschachtelt sind und welche Werte ihre Eigenschaften annehmen dürfen. Die Spezifikation selbst muss diesem Schema entsprechen. Dadurch lassen sich Struktur, Verweise und Konsistenz automatisch validieren. Eine Spezifikation wird also nicht nur geschrieben, sondern auch geprüft, ähnlich wie Quellcode gegen sein Typsystem.

Die Spezifikation wird so zur einzigen Quelle der Wahrheit (**Single Source of Truth**): Ein Spezifikationsmodell speist viele Sichten für unterschiedliche Zielgruppen. Dazu gehören navigierbare Dokumentation für Entwickler, verbindliche Dokumente für Kunden und maschinenlesbare Syntaxbäume für KI-Agenten. Diagramme lassen sich sogar direkt aus dem Modell ableiten, statt sie von Hand zu zeichnen, sodass sie immer zum Text passen. **SpecBook** ist ein Beispiel für diesen modernen Ansatz.

Fragen

- Was bedeutet **Single Source of Truth** im Kontext einer **Spezifikation**?

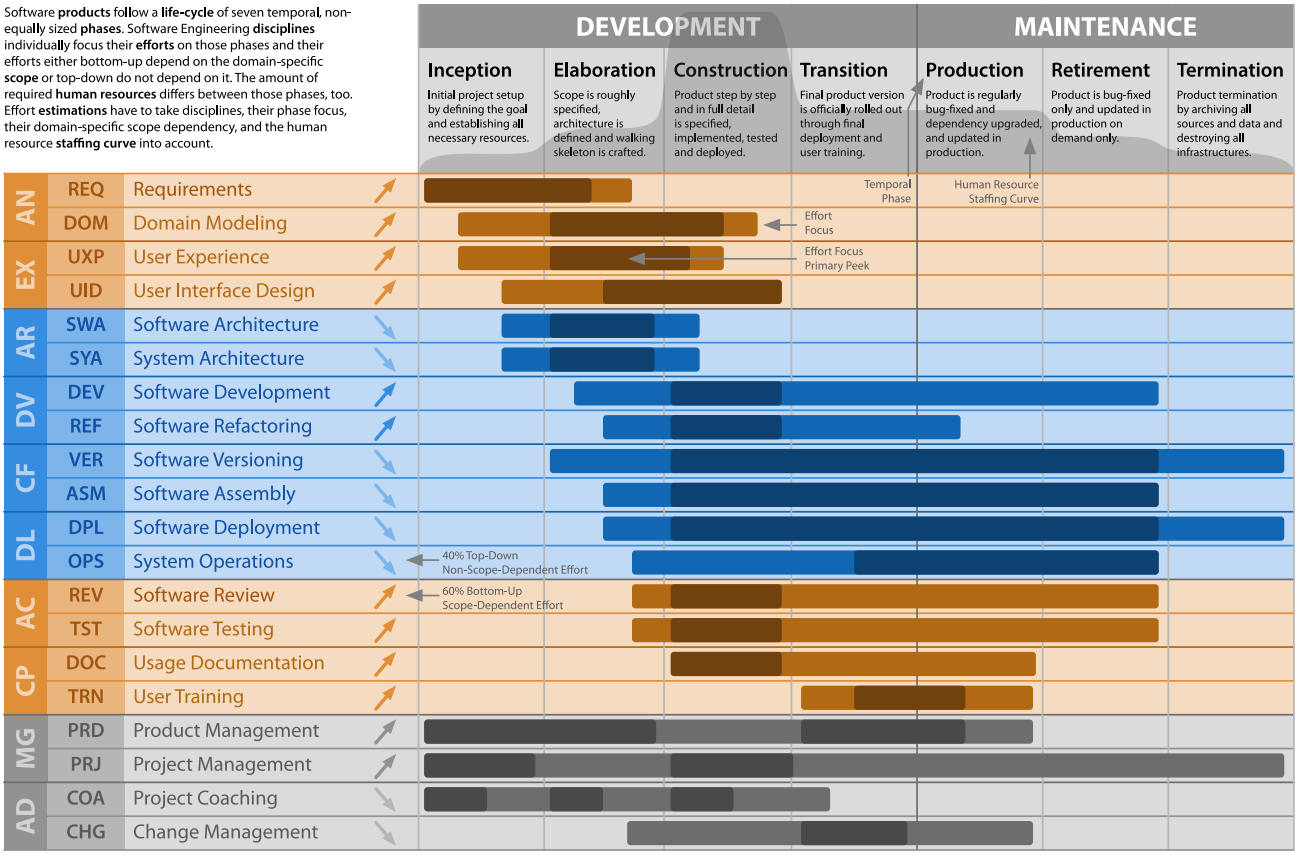
ENGINEERING FUNDAMENTALS

Software Engineering Efforts

TECHNISCHE UNIVERSITÄT MÜNCHEN

EF 03.1

Software **products** follow a **life-cycle** of seven temporal, non-equally sized **phases**. Software Engineering **disciplines** individually focus their **efforts** on those phases and their efforts either bottom-up depend on the domain-specific **scope** or top-down do not depend on it. The amount of required **human resources** differs between those phases, too. Effort **estimations** have to take disciplines, their phase focus, their domain-specific scope dependency, and the human resource **staffing curve** into account.



Intellectual Content: Version 1.0.4 (2021-04-11), Authored 2021-2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.1 (2021-04-11), Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com> – All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

phases & efforts

Softwareprodukte folgen einem **Lebenszyklus** von sieben zeitlichen, nicht gleich großen **Phasen**. Die einzelnen **Disziplinen** des Software-Engineering fokussieren ihre **Aufwände** auf diese Phasen und ihre Aufwände sind entweder von unten nach oben vom fachlichen Umfang abhängig oder von oben nach unten nicht davon abhängig. Der Umfang der erforderlichen **personellen Ressourcen** unterscheidet sich ebenfalls zwischen den Phasen.

Aufwandsschätzungen müssen die Disziplinen, ihre Phasenschwerpunkte, ihre Abhängigkeit vom fachlichen Umfang und die Personalbesetzungskurve berücksichtigen.

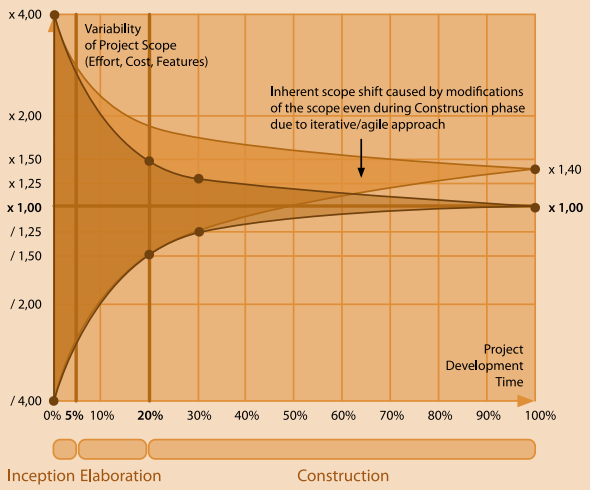
Die sieben sequenziellen Phasen stehen außerdem in keinerlei Konflikt mit agilen Vorgehensmodellen: Agile zeitliche Perioden (in Scrum “Sprints” genannt) unterteilen nämlich lediglich die einzelnen Phasen.

Fragen

- Wie wird die Software-Engineering-**Phase** genannt, welche den größten Personalbedarf besitzt und in der primär die fachlichen Funktionalitäten realisiert werden?

Uncertainty & Elaboration

Cone of Uncertainty



The ***Cone of Uncertainty*** (*Steve McConnell*, 2006) tells how the variability of the project scope (measured in Effort, Cost or Features) in Software Development changes over time. Initially, it usually is within the range of +/- 400% of the final scope.

The early development phases Inception and Elaboration especially have to ensure that within the first 20% of the project, the variability is reduced noticeably to just +/- 50%. During the initial iterations of the Construction phase within the first 30% of the project, the variability usually can be further reduced to about +/- 25%.

For iterative/agile approaches, experience showed that during the Construction phase inherently the final scope further shifts by about + 40% due to the just step-by-step learned required details of the required solution. This especially has to be taken into account for estimations.

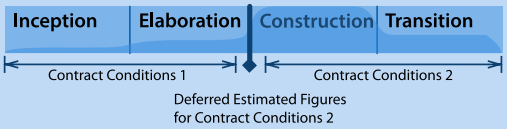
Essential Elaboration Phase

Walking Skeleton:

The ***Walking Skeleton*** (or *Technical Breakthrough*) is the design and implementation of the bare technical foundation of an application, still *without* any domain-specific functionalities. It is made during the Elaboration phase with the primary purpose to establish a stable integration of all technical aspects (libraries, frameworks, build procedures, etc) onto which the domain-specific functionalities later can be successively put onto.



Agile Fixed-Price Contracts:



The ***Agile Fixed-Price*** is an agile variant of a fixed-price contract, *not* a fixed-price project with an agile development process.



There are two important inherent aspects:
First, the contract contains two types of conditions: one (usually *Time & Material* but fixed duration based) for the Inception and Elaboration phases in order to make experiences and to gather necessary figures, and one (usually Fixed-User-Story and/or Fixed-Price based) for the Construction and Transition phases based on deferred estimated figures, gathered in the Elaboration phase.
Second, the Fixed-Price aspect of the contract is actually based on an amount of User-Stories (resulting in costs by multiplying them with either an average hourly rate of an engineer or individual rates based on engineer job levels), which the customer can 1:1 *exchange* during the project for different deliverables.

The crux of an Agile Fixed-Price contract is: first, during the Inception and Elaboration phases the supplier can shrink the *Cone of Uncertainty* and this way its risks dramatically, and second, during the Construction and Transition phases the customer still remains flexible in scope.

Intellectual Content: Version 1.0.1 (2021-10-10), Authored 2021-2020 by Dr. Ralf S. Engelschall based on concepts from Steve McConnell
Graphical Illustration: Version 1.0.1 (2021-10-10), Copyright © 2021-2020 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contents only.

Der **Konus der Unsicherheit** zeigt auf, wie sich die Variabilität des Projektumfangs (gemessen in Aufwand, Kosten oder Features) in der Softwareentwicklung im Laufe der Zeit ändert. Die frühen Entwicklungsphasen Inception und Elaboration müssen insbesondere dafür sorgen, daß sich die Variabilität merklich reduziert.

Bei iterativen/agilen Ansätzen zeigt die Erfahrung, daß sich in der Phase Construction naturgemäß der endgültige Umfang weiter verschiebt, weil die erforderlichen Details der gewünschten Lösung erst schrittweise gelernt werden.

Die Phase Elaboration ist insbesondere wichtig für die Erstellung des Technischen Durchstichs (**Walking Skeleton**), bei dem alle technischen Integrationen von Libraries, Frameworks, Build-Prozeduren, etc., durchgeführt werden, ohne bereits fachliche Funktionalitäten zu implementieren.

In **Agile-Fixed-Price**-Projektverträgen wird aufgrund des **Konus der Unsicherheit** in der Regel zwischen den frühen Phasen Inception und Elaboration und den Hauptphasen Construction und Transition unterschieden. Die Vertragsbedingungen der letzteren hängen in der Regel von Zahlen ab, die erst am Ende der Phase Elaboration seriös geschätzt werden können.

Fragen

- Was wird insbesondere in der Projektphase “Elaboration” entwickelt?

Effort Estimations

Feedback Content: Version 1.0.1 (2022-10-10), Authored 2021-2022 by Dr. Ralf S. Engelschall based on concepts from Steve McConnell
Graphical Illustration: Version 1.0.1 (2022-10-10), Copyright © 2021-2022 Dr. Ralf S. Engelschall <http://engelschall.com>, All rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Estimation & Variability

Three-Point Estimation and Estimation Variability Classes:

$e = (b + 4 \times m + w) / 6$	expected effort (weighted average)
$s = (w - b) / 6$	standard deviation (effort variation)

b: best-case (optimistic)
m: most-likely (realistic)
w: worst-case (pessimistic)

Insane Variability:	+/- 10%
Very Good Variability:	+/- 15%
Good Variability:	+/- 20%
Acceptable Variability:	+/- 25%

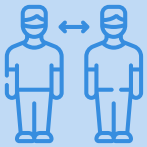


Conversion & Normalization

1. Ask Estimater:
"How many Person-Days do you need when you can focus on this task?"

2. Convert from Estimator to Performer:
(see also CAP model, http://cap-model.com)

Estimator \ Performer	Non-Linear Effort Reduction 0% (Novice)	10% (Practitioner)	25% (Master)	45% (Expert)	80% (Guru)
Novice	1,00	0,90	0,75	0,55	0,20
Practitioner	1,11	1,00	0,83	0,61	0,22
Master	1,33	1,20	1,00	0,73	0,27
Expert	1,82	1,64	1,36	1,00	0,36
Guru	5,00	4,50	3,75	2,75	1,00

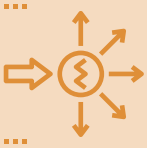


Sizes & Variability

Estimation Sizes and Estimation Variability:

T-Shirt-Size (Logically)	XXS	XS	S	M	L	XL	XXL	XXXL
Fibonacci-Size (PD or SP)	**0,50**	**1**	**2**	**3**	**5**	**8**	**13**	**21**
Size Variability (-)	0,25	0,25	0,50	0,50	1,00	1,50	2,50	4,00
Size Variability (+)	0,25	0,50	0,50	1,00	1,50	2,50	4,00	8,00

Notice: Estimations can be done in *Person-Days (PD)* or *Story-Points (SP)*. In both cases, keep in mind to use something like the *Fibonacci* numbers which increase in a non-linear fashion and express the increasing variability with the increasing total amount of estimated effort.



Risk Mitigation & Upscaling

3. Adjust for Reality:
Estimator Optimism: +30%
Performer Meetings: +20%

4. Adjust for Uncertainty:

Domain	Inception	Elaboration	Construction
unknown	30%	40%	20%
partially known	15%	20%	10%
fully known	0%	0%	0%

Technology	Inception	Elaboration	Construction
unknown	20%	60%	10%
partially known	10%	30%	5%
fully known	0%	0%	0%

Process	Inception	Elaboration	Construction
unknown	60%	40%	10%
partially known	30%	20%	5%
fully known	0%	0%	0%

People	Inception	Elaboration	Construction
unknown	60%	40%	0%
partially known	30%	20%	0%
fully known	0%	0%	0%



Aufwandsabschätzungen basieren in der Regel auf einer **Drei-Punkt-Schätzung**, bei der ein gewichteter Durchschnitt aus "best case", "most likely" und "worst case" verwendet wird. Eine gute Schätzung liegt in der Praxis bei etwa +/- 20%.

Bei **Expertenschätzungen** wird in der Praxis in der Regel eine feste Skala von Schätzgrößen verwendet, die auf der **Fibonacci**-Zahlenfolge basiert, um der Tatsache Rechnung zu tragen, daß höhere Schätzaufwände auch eine höhere Schätzvariabilität aufweisen.

Zusätzlich muß man in der Regel die Schätzung von Experten nachjustieren, um der Tatsache Rechnung zu tragen, daß unterschiedliche Fähigkeiten und Erfahrungen zwischen dem Schätzer und dem späteren Ausführenden existieren, der Schätzer den üblichen menschlichen Optimismus an den Tag legte und der Ausführende unvermeidliche Ablenkungen durch Kommunikation und Meetings in der Praxis erfahren wird.

Im Fall von Unsicherheit aufgrund der völlig unbekannten oder zumindest teilweise unbekannten Aspekte Domäne, Technologie, Prozess und Menschen müssen die geschätzten Gesamtaufwände der üblichen Projektphasen zusätzlich hochskaliert werden.

Fragen

- Welche Variabilität hat eine **gute** Schätzung in etwa?

ENGINEERING FUNDAMENTALS

Agentic Software Engineering

TECHNISCHE UNIVERSITÄT MÜNCHEN



Intellectual Content: Version 1.0.0 (2026-04-28). Authored 2026 by Dr. Ralf S. Engelschall. Graphical Illustration: Version 1.0.0 (2026-04-28). Copyright © 2026 Dr. Ralf S. Engelschall <http://engelschall.com>. All Rights Reserved. Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

concepts & relationships

Grundlagen: Software ist die immaterielle, veränderbare Gesamtheit von Programmen und Daten, die einen Computer Aufgaben ausführen lassen. Engineering ist die systematische, nachvollziehbare Anwendung wissenschaftlicher Erkenntnisse, um Lösungen zu entwerfen, zu bauen, zu betreiben und zu warten. Software Engineering wendet diese Disziplin auf den gesamten Lebenszyklus eines Softwaresystems an.

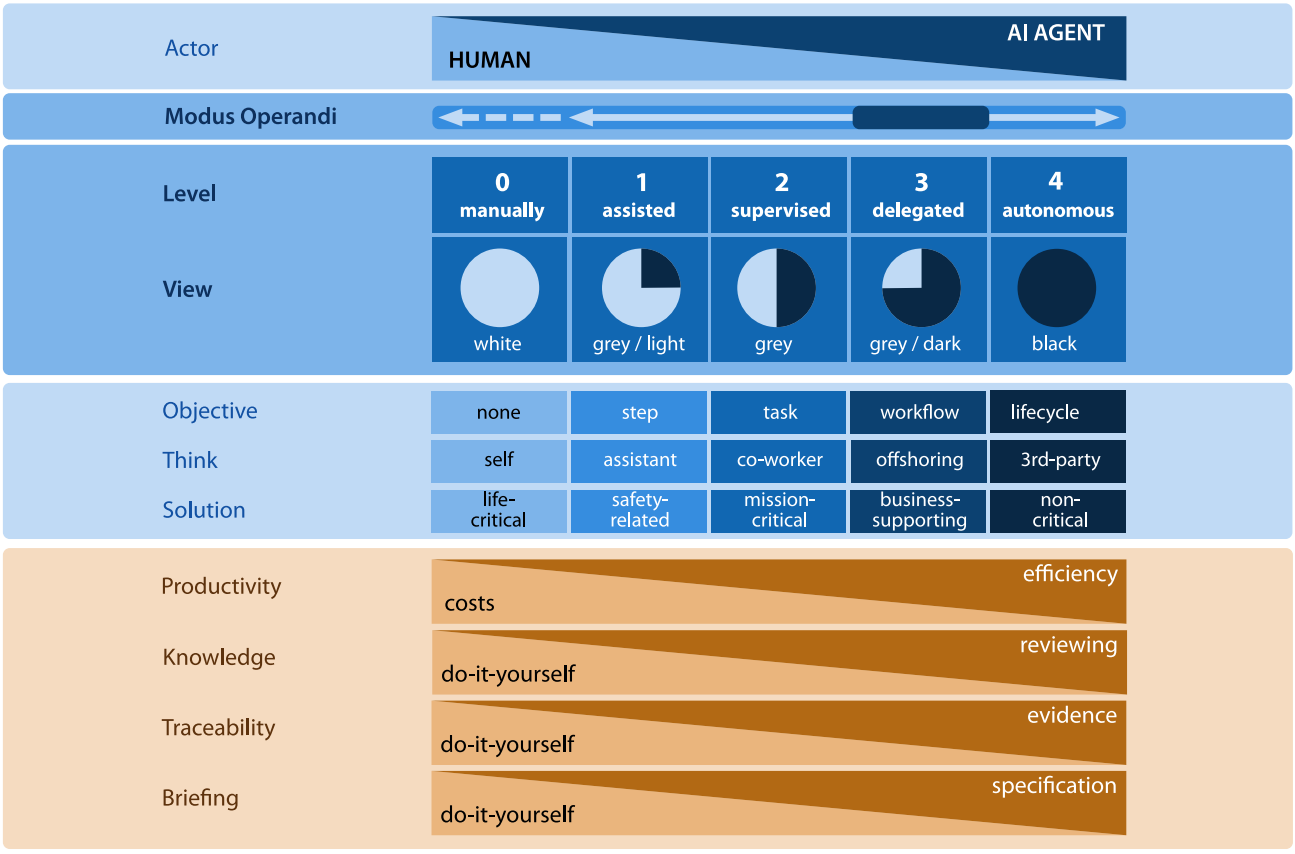
Von KI zu Agenten: Künstliche Intelligenz reicht von symbolischer KI bis zu maschinellem Lernen und generativen Modellen wie LLMs. Ein Software-Agent handelt autonom im Auftrag anderer, um übertragene Ziele zu erreichen. Er nimmt seine Umgebung über Eingaben wahr, handelt über Ausgaben und agiert dabei reaktiv, proaktiv und sozial. Ein KI-Agent ist ein solcher Agent, dessen Denkkern ein KI-Modell bildet, meist ein LLM. Der Agent-Harness um das Modell herum betreibt die Steuerungsschleife (z. B. ReAct), stellt Werkzeuge bereit, verwaltet den Kontext und setzt Leitplanken durch. Das Modell entscheidet, was getan wird; der Harness bestimmt, was überhaupt getan werden kann.

Agentic Software Engineering: Agentic AI baut Systeme aus autonomen Agenten, die planen, handeln und sich anpassen, um langfristige Ziele zu verfolgen. Dabei führen sie reale Aktionen aus, statt nur Antworten zu generieren. Im Software Engineering übernehmen Agenten mehrstufige Aufgaben über den gesamten Entwicklungszyklus hinweg, innerhalb eines nachvollziehbaren Prozesses. Menschliche Ingenieure konzentrieren sich zunehmend darauf, Ziele zu spezifizieren, die Arbeit zu überwachen und zu prüfen und sie über Human-in-the-Loop-Freigaben zu steuern.

Fragen

- Welche Aufgabenteilung besteht bei einem AI-Agenten zwischen dem AI-Model und dem Agent-Harness?

Agentic Modus Operandi



Intellectual Content, Version 1.0.0 (2025-12-28), Authored 2026 by Dr. Ralf S. Engelschall
Graphical Illustration, Version 1.0.0 (2025-12-28), Copyright © 2026 Dr. Ralf S. Engelschall <http://engelschall.com> – All Rights Reserved
Unauthorized Reproduction Prohibited, Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only

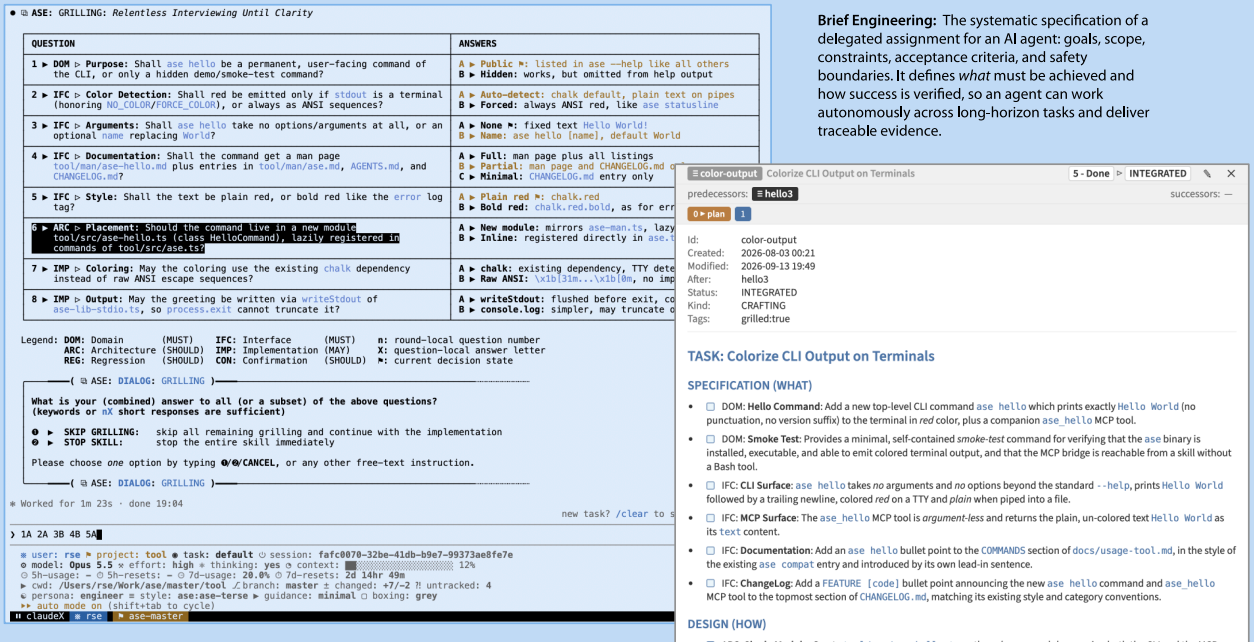
Aufgaben werden zwischen Mensch und AI-Agent über fünf Autonomiestufen aufgeteilt: manuell (0), assistiert (1), überwacht (2), delegiert (3) und autonom (4). Jede Stufe entspricht einer vertrauten Arbeitsweise: es selbst machen, einen Assistenten einsetzen, mit einem Kollegen zusammenarbeiten, Offshoring und schließlich die Übergabe an einen Dritten. Dabei wandelt sich das Arbeitsergebnis des Agenten von einer White Box (vollständig sichtbar und nachvollziehbar) zu einer Black Box (intransparent).

Mit jeder Stufe übernimmt der Agent ein größeres Ziel: von keinem über einen einzelnen Schritt, eine Aufgabe und einen Workflow bis hin zum gesamten Lebenszyklus. Höhere Autonomie passt zu weniger kritischen Lösungen, von lebenskritischen und sicherheitsrelevanten Systemen am manuellen Ende bis zu unterstützenden und unkritischen Systemen am autonomen Ende.

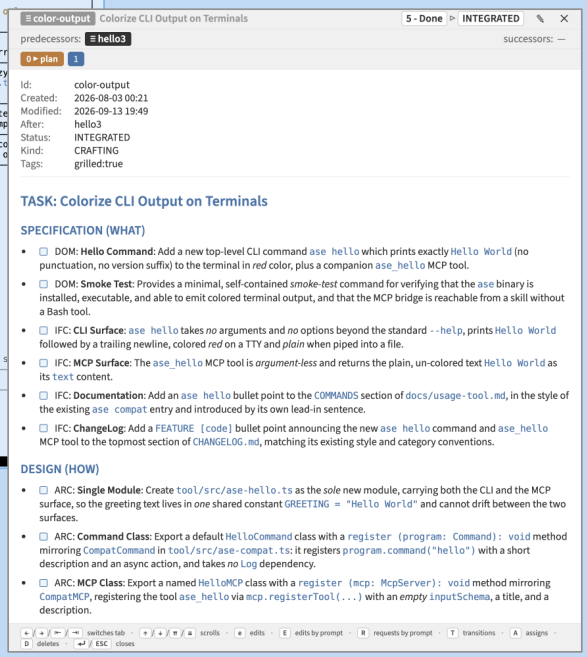
Der Weg zur Autonomie verschiebt die Produktivität von menschlichen Kosten hin zu Effizienz. Auch die Rolle des Menschen verändert sich: Wissen verlagert sich vom Selbermachen zum Reviewen, Nachvollziehbarkeit vom Selbermachen zu Nachweisen und das Briefing vom Selbermachen zu präziser Spezifikation.

Fragen

- Auf welcher Autonomiestufe arbeitet ein Agent, der selbstständig einen kompletten Workflow umsetzt?



Brief Engineering: The systematic specification of a delegated assignment for an AI agent: goals, scope, constraints, acceptance criteria, and safety boundaries. It defines *what* must be achieved and how success is verified, so an agent can work autonomously across long-horizon tasks and deliver traceable evidence.



Prompt Engineering: The craft of formulating individual inputs (prompts) to an AI model so that a single request reliably produces the desired output. It focuses on phrasing, structure, examples, and instructions within one prompt–response exchange.

Context Engineering: The systematic design and management of everything an AI model sees while it works: instructions, retrieved knowledge, tool definitions and results, memory, and conversation history. It decides what goes into the limited context window at each step of an agent's control loop, usually handled by the Agent Harness.

Drei Ebenen des Engineerings: Prompt Engineering optimiert einen einzelnen Prompt-Antwort-Austausch. Context Engineering steuert alles, was das Modell in jedem Schritt der Steuerungsschleife des Agenten sieht. Brief Engineering spezifiziert einen vollständigen delegierten Auftrag: was erreicht werden muss und wie der Erfolg überprüft wird.

Grilling für Klarheit: "Grilling" bedeutet, dass der Agent den Menschen beharrlich befragt, bis das Briefing klar ist. In der Praxis wird er meist angewiesen, strukturierte Multiple-Choice-Fragen zu stellen, priorisiert nach Domäne, Schnittstelle, Architektur und Implementierung.

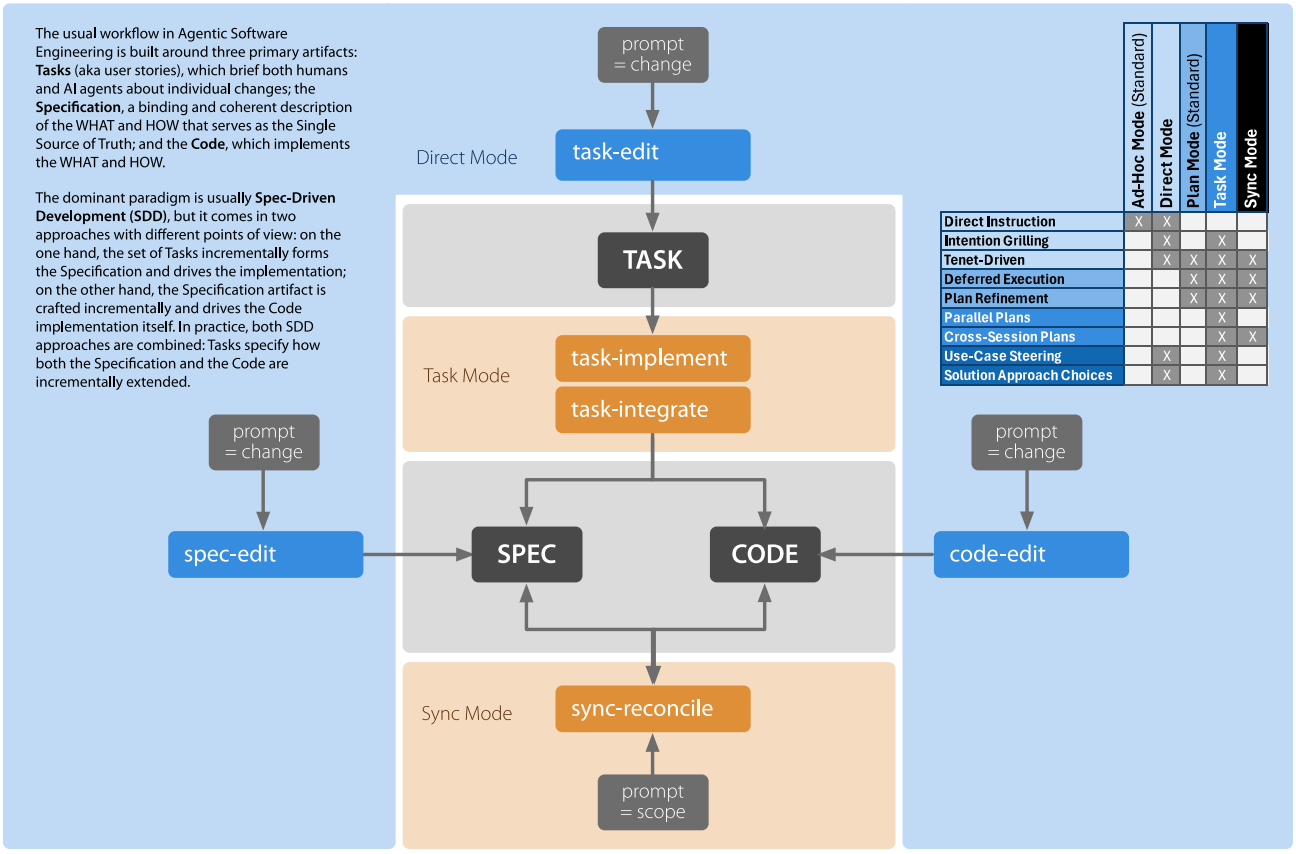
Die Antworten werden meist zu einer strukturierten Aufgabe mit abhakbaren Punkten, gegliedert in eine Spezifikation (WAS), ein Design (WIE) und eine Verifikation (WANN). So erhält der Agent ein präzises Briefing für autonomes Arbeiten und der Mensch eine nachvollziehbare Grundlage für das Review.

Fragen

- Was versteht man unter "Grilling", und warum ist es gerade bei hoher Autonomie des Agenten wichtig?

Intellectual Content: Version 1.0.0 (2026-08-28) Authored 2026 by Dr. Ralf T. Kugler
Graphical Illustration: Version 1.0.0 (2026-08-28) Copyright © 2026 Dr. Ralf T. Kugler
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.
All Rights Reserved.

Agentic Workflow



Der übliche Workflow im Agentic Software Engineering basiert auf drei zentralen Artefakten: den **Tasks** (auch User Stories genannt), die sowohl Menschen als auch AI-Agenten über einzelne Änderungen briefen; der **Spezifikation**, einer verbindlichen und kohärenten Beschreibung des WAS und WIE, die als Single Source of Truth dient; und dem **Code**, der das WAS und WIE implementiert.

Das vorherrschende Paradigma ist meist **Spec-Driven Development (SDD)**, das jedoch in zwei Ansätzen mit unterschiedlichen Blickwinkeln auftritt: Einerseits bildet die Menge der Tasks schrittweise die Spezifikation und treibt die Implementierung voran; andererseits wird das Spezifikationsartefakt schrittweise ausgearbeitet und treibt selbst die Implementierung des Codes voran. In der Praxis werden beide SDD-Ansätze kombiniert: Tasks legen fest, wie sowohl die Spezifikation als auch der Code schrittweise erweitert werden.

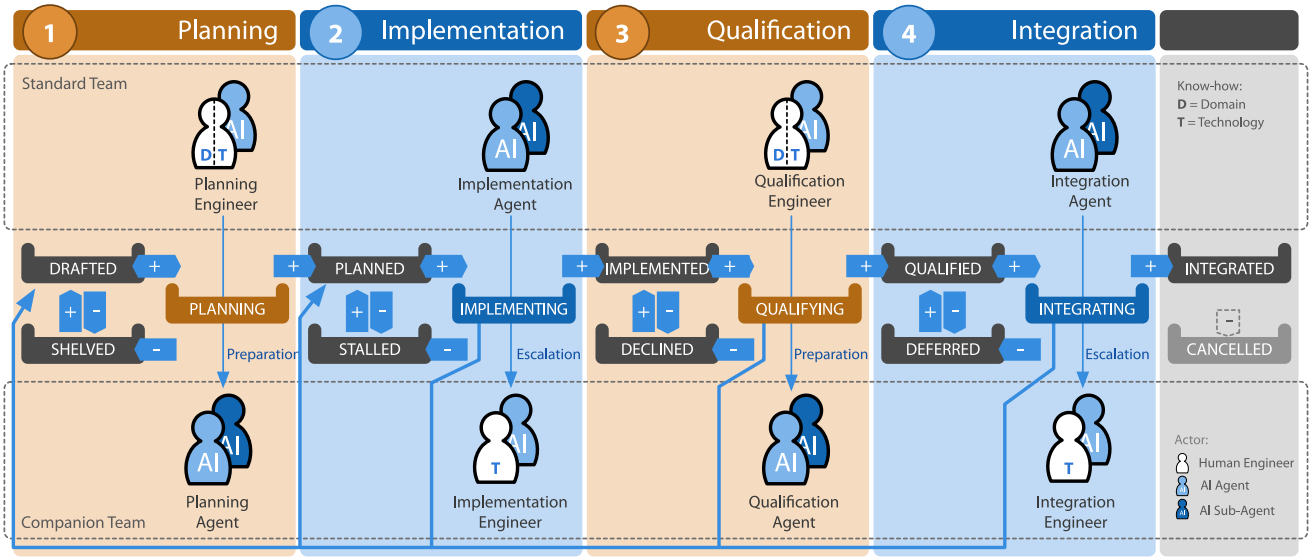
Fragen

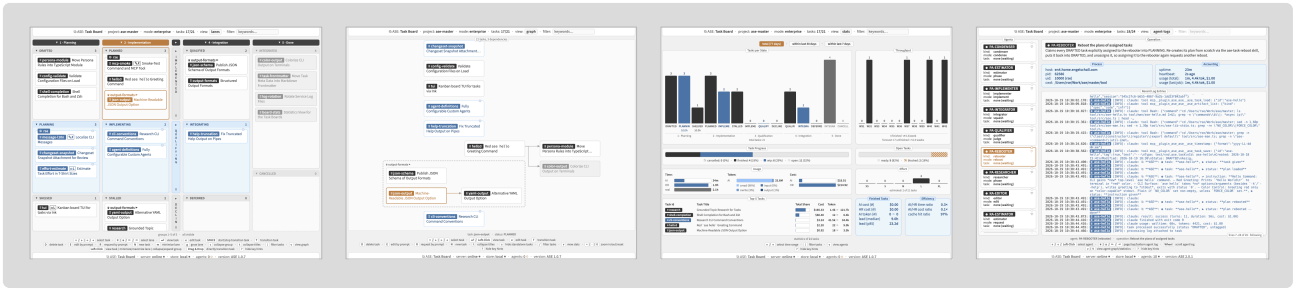
- Welche drei zentralen Artefakte gibt es im Agentic Software Engineering, und wie werden die beiden SDD-Ansätze in der Praxis kombiniert?

ENGINEERING FUNDAMENTALS

Agentic Development Pipeline

TECHNISCHE UNIVERSITÄT MÜNCHEN





Ein Task durchläuft vier Phasen: Planung, Implementierung, Qualifizierung und Integration. Seine Hauptzustände sind DRAFTED, PLANNED, IMPLEMENTED, QUALIFIED und INTEGRATED, verbunden durch die aktiven Zustände PLANNING, IMPLEMENTING, QUALIFYING und INTEGRATING. Jede Phase hat zudem einen Parkzustand (SHELVED, STALLED, DECLINED, DEFERRED), Tasks können abgebrochen werden (CANCELLED), und fehlgeschlagene Tasks springen in frühere Phasen zurück.

Das Standard-Team besteht aus Menschen und AI-Agenten, die sich über die Phasen hinweg abwechseln. Menschliche Engineers mit Domänen- und Technologie-Know-how übernehmen Planung und Qualifizierung, wo es auf das Urteil über das WAS und dessen Abnahme ankommt. AI-Agenten übernehmen Implementierung und Integration, wo die Umsetzung des WIE im Vordergrund steht.

Das Companion-Team ergänzt diesen Aufbau: Jedem primären Akteur steht ein Gegenpart zur Seite. AI-Agenten übernehmen die Vorbereitung für die menschlichen Planning und Qualification Engineers, während menschliche Engineers mit Technologie-Know-how Eskalationen der Implementation und Integration Agents übernehmen.

Fragen

- Warum übernehmen im Standard-Team Menschen die Planung und Qualifizierung, AI-Agenten dagegen die Implementierung und Integration?

EF 05.5

Textual content: Version 1.0.0 (2026-04-28). Authored 2026 by Dr. Ralf D. Engelken.
Gedankenfabrik Iteration, Version 1.0.0 (2026-04-28). Copyright © 2026 Dr. Ralf D. Engelken. (http://creativecommons.org/licenses/by/4.0/). Reproduction permitted, attribution required. Image contents only.

tasks & states